

Learning Report

1. Why ViT?

In a CNN, each convolutional filter only sees a small neighbourhood of pixels. To build up an understanding of the full image, the network has to stack many layers and rely on pooling operations to gradually widen the receptive field. A Vision Transformer works differently -- it processes the entire image at once by letting every region attend to every other region through self-attention.

The original ViT paper (Dosovitskiy et al., 2020) showed that when trained on enough data, this approach matches or outperforms CNNs. One tradeoff is that ViTs tend to be data-hungry; they generally need larger datasets to train well compared to CNNs, which benefit from their built-in spatial priors. However, with a pretrained model like DINOv2, we get the benefits without needing to train from scratch ourselves.

For our galaxy morphology work, ViT is a natural fit because galaxy structure can span the entire image -- a spiral arm on one side relates to the core in the centre. Self-attention handles that kind of long-range relationship directly.

2. Patchification

Transformers were originally designed for sequences of words, so images need to be converted into a sequence of tokens first. ViT does this by slicing the image into a regular grid of non-overlapping square patches.

In DINOv2-base the patch size is 14x14 pixels. Our galaxy images start at 160x160 but are resized to 224x224 by the image processor before entering the model. That gives $224 / 14 = 16$ patches per side, or $16 \times 16 = 256$ patches total.

Internally this is implemented as a single convolution layer with both kernel size and stride set to 14. It is not a standard feature-extracting convolution -- its only job is to slice and linearly project each patch in one operation. After this step, the 2-D spatial layout is flattened into a 1-D sequence that the Transformer can consume.

3. Patch Embeddings

Each 14x14x3 patch is projected into a 768-dimensional vector through the convolution layer mentioned above. These vectors are called patch embeddings. With 256 patches, we get a matrix of size 256 x 768.

The 768 numbers in each embedding do not map one-to-one to specific visual properties like 'brightness' or 'edge orientation'. They are learned distributed representations -- the model decides during training how to encode visual information across all 768 dimensions. This is similar to how word embeddings in NLP encode meaning across hundreds of dimensions rather than in single entries.

Because the projection is learned, DINOv2 can encode low-level features like colour and texture as well as higher-level patterns, depending on what is most useful for the self-supervised training objectives.

4. Positional Embeddings

Self-attention treats input tokens as an unordered set. Without any additional information, swapping two patch embeddings in the sequence would produce the same attention output. This is a problem because spatial location clearly matters -- a bright patch in the centre of a galaxy image means something very different from a bright patch at the edge.

To solve this, a learnable positional embedding is added to each patch embedding before the Transformer layers. There are 257 of these vectors (one for each of the 256 patches plus the CLS token), each with 768 dimensions.

The model learns them during pretraining.

An interesting detail is that these positional embeddings can be interpolated to handle images of different sizes at inference time, though in our case we stick with the standard 224x224 input. DINOv2 handles all of this internally, so we did not need to set up positional embeddings manually.

5. CLS Token

The CLS token is borrowed from BERT in NLP. It is a learnable vector that gets prepended to the patch sequence, bringing the total from 256 to 257 tokens. Unlike the patch tokens, it does not correspond to any spatial region of the image.

As the sequence moves through each Transformer layer, the CLS token collects information from all patch tokens via self-attention. By the final layer, it has aggregated enough context to serve as a single summary of the entire image.

In our experiment the output tensor had shape [1, 257, 768]. We sliced out the first token to get the CLS embedding with shape [1, 768]. An alternative approach would be to average all 256 patch tokens (global average pooling), but the CLS token is the standard choice for DINOv2 and is what the model was trained to optimise.

6. ViT Transformer Architecture

The core of ViT is a stack of Transformer encoder blocks. DINOv2-base uses 12 such blocks. Each block has two sub-layers: Multi-Head Self-Attention (MSA) followed by a feed-forward MLP. Both are wrapped with residual connections and Layer Normalisation.

In the MSA sub-layer, the 768-d token embeddings are split across 12 attention heads ($768 / 12 = 64$ dimensions per head). Each head independently computes attention weights between all pairs of tokens and produces a weighted combination. The outputs of all heads are concatenated back to 768 dimensions.

The MLP sub-layer applies two linear transformations with a GELU activation in between. It expands the dimension from 768 to 3072 (a 4x expansion) and projects back down. While self-attention mixes information across tokens, the MLP processes each token independently and transforms its feature representation.

The total model size is about 86 million parameters. DINOv2 uses the pre-norm variant, meaning LayerNorm is applied before each sub-layer rather than after, which tends to make training more stable.

7. CLS vs Patch Tokens

After the final Transformer block, we have two types of output that serve different purposes.

The CLS token [1, 768] captures the image as a whole. It is suited for tasks where we need a single descriptor per image, such as classifying galaxy morphology or searching for similar-looking galaxies in a database.

The patch tokens [1, 256, 768] retain spatial structure. Each token still corresponds to its original 14x14 region. These are useful when we need to localise features -- for instance, identifying where the spiral arms are, or segmenting the bulge from the disk.

In our notebook we used both: the CLS token for t-SNE clustering across 100 galaxies, and the patch tokens for PCA-based spatial feature maps. Having both from a single forward pass is convenient.

8. What is DINOv2?

DINOv2 (Oquab et al., 2023) is a self-supervised vision model from Meta AI. The key idea is that the model learns representations without any human-annotated labels. It was trained on a curated dataset of 142 million images called LVD-142M.

The training uses two objectives together. Self-distillation (from the original DINO method) trains a student network to match the outputs of a momentum teacher on different crops and augmentations of the same image. Masked image modelling (from iBOT) hides random patches and asks the student to recover what the teacher would have predicted for them.

Compared to the earlier DINO (v1), DINOv2 made significant improvements in data curation, training stability, and the addition of the masking objective. The result is a model whose features generalise well across many tasks without any task-specific fine-tuning, which is exactly what we rely on in our project.

9. What does DINOv2 output?

A forward pass through DINOv2-base with a single 224x224 image returns a tensor called `last_hidden_state` with shape `[1, 257, 768]`. The 257 dimension breaks down into 1 CLS token + 256 patch tokens.

We separated these into CLS `[1, 768]` and patches `[1, 256, 768]`. There is no classification head attached -- the model only provides raw embeddings, and it is up to us to add any task-specific layer on top.

The model can also return hidden states from all 12 intermediate layers if we pass `output_hidden_states=True`. Earlier layers tend to capture low-level features like edges, while later layers encode more abstract, semantic information. For now we only used the final layer outputs, but the intermediate states could be useful for multi-scale analysis later.

10. How will DINOv2 be used in this project?

We are using DINOv2 as a frozen feature extractor. This means we do not update any of the model's 86 million parameters -- we simply pass each galaxy image through and collect the output embeddings. This is a form of transfer learning: features learned from 142 million natural images turn out to work surprisingly well on astronomical data.

With the CLS embeddings, we ran t-SNE and PCA on 100 galaxies and observed that the model groups visually similar morphologies together, even though it has never been trained on galaxy images. This suggests the representations are general enough to be useful for our domain.

With the patch embeddings, we built spatial feature maps using PCA to visualise what regions of each galaxy contribute differently to the model's representation. Going forward, the plan is to train a lightweight classifier (like a linear probe or k-NN) on top of these frozen CLS embeddings using the `gz10_label` annotations provided in the dataset.